МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Отчет по лабораторной работе №3

по курсу «Операционные системы»

Тема: «Многопоточная четно-нечетная сортировка Бетчера»

Выполнил(а): А. Е. Пронина
Группа: М8О-203БВ-25
Преподаватель: Е. С. Миронов

Москва, 2026

Условие

Цель работы: приобретение практических навыков управления потоками операционной системы и обеспечения синхронизации между ними.

Задание: разработать программу для многопоточной обработки данных с использованием стандартных средств Windows или Unix. Максимальное количество потоков, одновременно работающих в программе, задается аргументом командной строки. Необходимо показать потоки средствами операционной системы и исследовать зависимость ускорения и эффективности от размера данных и числа потоков.

Вариант 5: отсортировать массив целых чисел при помощи четно-нечетной сортировки Бетчера.

Метод решения

Общая идея алгоритма

Используется сортировочная сеть Бетчера (odd-even mergesort network). В отличие от сортировок, которые выбирают следующие сравнения в зависимости от значений элементов, сеть заранее определяет пары *позиций* для сравнения. Компаратор сравнивает два числа и при необходимости меняет их местами, чтобы меньшее значение оказалось слева.

Алгоритм рекурсивно сортирует две половины массива, затем сливает их с помощью четно-нечетной схемы: отдельно формируются сравнения для чередующихся позиций и выполняются завершающие сравнения соседних пар. Полученные операции можно сгруппировать по *уровням*. На одном уровне ни одна позиция не используется двумя компараторами одновременно, а следующий уровень зависит от результата предыдущего.

Для длины $n = 2^m$ число уровней классической сети составляет $m(m+1)/2$, общее число сравнений имеет порядок $O(n \log^2 n)$. Глубина сети $O(\log^2 n)$ позволяет выполнять независимые сравнения одного уровня параллельно.

Пример построения сети

Рассмотрим восемь элементов:

$$[8, 3, 7, 1, 6, 2, 5, 4].$$

Ниже номера позиций указаны *с единицы*. В таблице приведены пары, сравниваемые на каждом уровне, и состояние массива после него.

Уровень	Сравниваемые позиции	Массив после уровня
1	(1,2), (3,4), (5,6), (7,8)	3, 8, 1, 7, 2, 6, 4, 5
2	(1,3), (2,4), (5,7), (6,8)	1, 7, 3, 8, 2, 5, 4, 6
3	(2,3), (6,7)	1, 3, 7, 8, 2, 4, 5, 6
4	(1,5), (2,6), (3,7), (4,8)	1, 3, 5, 6, 2, 4, 7, 8
5	(3,5), (4,6)	1, 3, 2, 4, 5, 6, 7, 8
6	(2,3), (4,5), (6,7)	1, 2, 3, 4, 5, 6, 7, 8

На последнем уровне сравниваются позиции (2,3), (4,5) и (6,7), а не произвольные соседние элементы: именно эти пары остаются после рекурсивного четно-нечетного слияния. Например, значения 3 и 2 меняются местами. Внутри одного уровня сравнения независимы и могут быть распределены между потоками.

Произвольная длина массива и экономия памяти

Программа принимает размер массива, не обязательно равный степени двойки. Для построения расписания используется логика полной сети Бетчера; сравнения, которые обращались бы к отсутствующим позициям, пропускаются. Это соответствует мысленному дополнению последовательности фиктивными значениями $+\infty$, но без выделения дополнительного массива для них.

Компараторы не сохраняются в отдельном большом списке. Пара индексов вычисляется непосредственно на текущем уровне сети. Поэтому дополнительная память не увеличивается пропорционально числу всех компараторов. Для входного массива по-прежнему требуется $O(n)$ памяти, а служебные данные потоков занимают $O(P)$, где P — число участников вычисления.

Распределение работы между потоками

Реализация выполняется на C++ под Ubuntu/WSL с использованием POSIX Threads (`pthread.h`). Основной поток участвует в вычислениях наряду с рабочими. Если лимит равен P, дополнительно создается не более $P-1$ потоков. Для каждого уровня сеть назначает каждому исполнителю собственные позиции исходного массива. Один поток не изменяет элементы, с которыми одновременно работает другой компаратор того же уровня.

Количество рабочих потоков фиксируется на время сортировки: потоки не создаются заново для каждого компаратора. Это уменьшает накладные расходы на их создание и завершение. При $P = 1$ сортировка осуществляется только главным потоком.

Синхронизация и обработка ошибок

Для согласования этапов используется `pthread_barrier_wait()`: каждый участник должен завершить сравнения текущего уровня перед переходом к следующему. Без этого один поток мог бы прочитать элемент до того, как его изменит другой поток на предыдущем уровне.

Помимо барьера, используются мьютекс и условная переменная для безопасного начала работы. Созданные потоки ожидают разрешения главного потока. Если создание очередного потока завершилось ошибкой, запуск отменяется: иначе уже созданные потоки могли бы бесконечно ждать недостающего участника барьера.

Главный поток вызывает `pthread_join()` для созданных рабочих потоков. После их завершения освобождаются служебная память и объекты синхронизации. Коды возврата функций POSIX должны проверяться; при обнаружении ошибки программа сообщает о ней. В частности, POSIX-функции семейства `pthread_*` обычно возвращают код ошибки напрямую, поэтому для диагностики можно применять `strerror(rc)`.

Описание программы

Проект организован по модулям:

- `src/main.cpp` — обработка аргументов, выделение и заполнение массива, запуск сортировки, вывод;
- `src/batcher.h`, `src/batcher.cpp` — формирование и исполнение компараторов сети Бетчера;
- `src/threads.h`, `src/threads.cpp` — потоки POSIX, распределение сравнений и синхронизация;
- `src/benchmark.h`, `src/benchmark.cpp` — измерение прошедшего времени через `clock_gettime()`;
- `CMakeLists.txt` — автоматизированная сборка с подключением библиотеки потоков.

Программа запускается командой `./lab3 N P`, где `N` — число элементов массива, `P` — максимальное число потоков, включая главный. Для небольших массивов печатается исходная и отсортированная последовательность; дополнительно выводятся PID, TID, результат проверки правильности и время сортировки.

Ключевые фрагменты реализации

Сравнение двух элементов сети (сокращенный смысловой фрагмент из `batcher.cpp`):

```
if (needCompare && arr[i] > arr[i + k]) {
    int temp = arr[i];
```

```
    arr[i] = arr[i + k];
    arr[i + k] = temp;
}
```

Здесь `i` — индекс первого элемента, `k` — расстояние до второго, а `needCompare` означает, что эта пара относится к текущему уровню сети. Значение `needCompare` определяется параметрами уровня и позицией элемента, а не содержимым массива.

Барьер после каждого уровня (`threads.cpp`):

```
compareStage(arr, n, p, k, id, threadCount);
int rc = pthread_barrier_wait(&barrier);
if (rc != 0 && rc != PTHREAD_BARRIER_SERIAL_THREAD) {
    // Error handling
}
```

Смысл фрагмента — не допустить начало следующего зависимого уровня, пока все участники не завершили текущий. Возвращаемое значение `PTHREAD_BARRIER_SERIAL_THREAD` также соответствует успешному ожиданию барьера.

Создание и ожидание рабочих потоков (`threads.cpp`):

```
int rc = pthread_create(&threads[i], nullptr,
                        worker, &data[i]);
// Check rc before counting the thread as created.

rc = pthread_join(threads[i], nullptr);
// Check rc and release resources after completion.
```

Параметры потока передаются через индивидуальную структуру, содержащую его номер и указатель на общие данные. Общие данные включают массив, число участников и объекты синхронизации. Такое разделение не дублирует массив для каждого потока.

Результаты работы

Проверка корректности

Программа была запущена в Ubuntu через WSL, из терминала VS Code. Для демонстрации выбран массив из 13 целых чисел и лимит 4 потока. Полученный фактический вывод:

```
$ ./lab3 13 4
PID: 20186
Array size: 13
Threads: 4
Before: 2946 6373 -5930 6206 1454 9876 -5172
        -8821 -6332 -436 2143 2175 7593
Main TID: 20186
Worker 1 TID: 20187
Worker 2 TID: 20188
Worker 3 TID: 20189
After: -8821 -6332 -5930 -5172 -436 1454
       2143 2175 2946 6206 6373 7593 9876
```

```
Sorting: OK
Time: 0.00185605 seconds
```

Полученная последовательность отсортирована по неубыванию. Размер 13 не равен степени двойки; следовательно, данный запуск одновременно демонстрирует обработку некратного степени двойки входа. Отображены четыре идентификатора потоков, включая главный. Измеренное время выполнения в этом запуске составило $0{,}00185605$ с.

Исследование ускорения и эффективности

Для исследования производительности проведены запуски на массивах из 1 000, 10 000, 100 000 и 1 000 000 элементов при $P = 1, 2, 4, 8$ потоках. Для каждого сочетания параметров программа выполнялась три раза. Во всех 48 запусках проверка завершилась сообщением `Sorting: OK`. В таблице приведены медианы трех измерений времени.

Ускорение и эффективность вычислялись по формулам

$$S(P) = \frac{T_1}{T_P}, \qquad E(P) = \frac{S(P)}{P} \cdot 100\%,$$

где T_1 и T_P — медианные времена сортировки массива одинакового размера соответственно с одним и P потоками. При каждом запуске программа генерировала одинаковые исходные данные с фиксированным начальным значением генератора случайных чисел.

Размер массива	Потоки P	Время T_P, с	Ускорение $S(P)$	Эффективность $E(P)$
1 000	1	0,000358	1,000	100,0%
1 000	2	0,003674	0,097	4,9%
1 000	4	0,007314	0,049	1,2%
1 000	8	0,014231	0,025	0,3%
10 000	1	0,006591	1,000	100,0%
10 000	2	0,011342	0,581	29,1%
10 000	4	0,015263	0,432	10,8%
10 000	8	0,025079	0,263	3,3%
100 000	1	0,100098	1,000	100,0%
100 000	2	0,070158	1,427	71,3%
100 000	4	0,055363	1,808	45,2%
100 000	8	0,055544	1,802	22,5%
1 000 000	1	1,322540	1,000	100,0%
1 000 000	2	0,775840	1,705	85,2%
1 000 000	4	0,625560	2,114	52,9%
1 000 000	8	0,374043	3,536	44,2%

На массивах из 1 000 и 10 000 элементов минимальное время показал один поток. Для 100 000 элементов самый быстрый результат достигнут при четырех потоках ($0{,}055363$ с), а для 1 000 000 элементов — при восьми ($0{,}374043$ с).

Идентификаторы потоков

В контрольном запуске процесс имел PID 20186. Главный поток имел TID 20186, а рабочие потоки — TID 20187, 20188 и 20189. Таким об-

разом, программа создала три дополнительных потока и использовала главный поток как четвертого участника сортировки.

Анализ результатов

Серия экспериментов показала, что влияние числа потоков существенно зависит от размера массива. При $N = 1\,000$ сортировка одним потоком заняла $0{,}000358$ с, тогда как восемью потоками — $0{,}014231$ с. При $N = 10\,000$ ситуация аналогична: один поток выполнял сортировку за $0{,}006591$ с, а восемь — за $0{,}025079$ с. На малых данных затраты на создание потоков и ожидание барьеров превышают выигрыш от параллельного выполнения сравнений.

При $N = 100\,000$ многопоточность уже дает ускорение. Время уменьшилось с $0{,}100098$ с на одном потоке до $0{,}055363$ с на четырех, то есть в $1{,}808$ раза. Увеличение числа потоков до восьми не улучшило результат: медианное время составило $0{,}055544$ с. Дополнительные участники увеличивают расходы на синхронизацию и конкурируют за вычислительные ресурсы.

На массиве из $1\,000\,000$ элементов быстрее всего работали восемь потоков: $0{,}374043$ с против $1{,}322540$ с для одного потока. Ускорение составило $3{,}536$, а эффективность — $44{,}2\%$. Максимальная эффективность среди многопоточных запусков для этого размера достигнута при двух потоках ($85{,}2\%$), хотя минимальное абсолютное время показали восемь потоков. Таким образом, наилучшее время и наибольшая эффективность использования каждого потока не всегда достигаются при одном и том же значении P.

Ускорение ограничивается необходимостью синхронизации после каждого уровня сети, накладными расходами потоков и доступом к общей памяти. При увеличении массива доля собственно вычислений возрастает, поэтому использование нескольких потоков становится выгоднее. Проверки `Sorting: OK` во всех 48 запусках подтверждают корректность порядка выходных элементов для исследованных размеров и числа потоков.

Выводы

В лабораторной работе реализована многопоточная четно-нечетная сортировка Бетчера на C++ с использованием POSIX Threads в Ubuntu/WSL. Сортировка принимает массив произвольной длины, вычисляет необходимые пары индексов без хранения всей сети компараторов и выполняет независимые сравнения несколькими потоками.

Было изучено создание потоков через `pthread_create()`, ожидание их завершения через `pthread_join()` и синхронизация с помощью барьера. Мьютекс и условная переменная обеспечивают согласованный за-

пуск рабочих потоков. Установленный пользователем предел числа потоков включает главный поток, который также участвует в вычислениях.

В ходе испытаний выполнено 48 запусков на четырех размерах массива и четырех значениях числа потоков. Во всех случаях сортировка завершилась корректно. Для небольших массивов эффективнее последовательное выполнение: накладные расходы на многопоточность превосходят пользу от параллельных сравнений. Для массива из 100 000 элементов лучшее время достигнуто при четырех потоках, а для массива из 1 000 000 элементов — при восьми. На самом большом массиве получено ускорение $3{,}536$ раза относительно однопоточного выполнения.

Эксперименты показали, что количество потоков следует оценивать вместе с размером данных: увеличение параллелизма ускоряет большие вычисления, но может замедлить обработку небольших массивов из-за стоимости синхронизации и управления потоками.

Список литературы

1. Сеть обменной сортировки со слиянием Бэтчера. Habr. `https://habr.com/ru/articles/275889/`.